Reinforcement Learning for Minesweeper

Implementing and Comparing SARSA(λ), Q-Learning, and Monte Carlo for Minesweeper

Fangteng Fu Yenchi Tseng Jiaqi Li

Spring 2026

1 Introduction

In this project we ask whether a carefully engineered *linear* reinforcement-learning agent can still perform strongly on this task. To answer this question, we implement a custom Minesweeper environment, train SARSA(λ), Q-Learning, and Monte Carlo agents with a shared feature representation, and evaluate how reward shaping, action masking, exploration scheduling, and inference-based features affect performance. Two contributions are especially central to our final system: **a decay+adaptive ε strategy** that reduces risky late-game exploration, and **feature engineering**.

The remainder of the report is organised as follows. Section 2 formalises the problem and reviews related work. Section 3 describes the environment, algorithms, and evaluation protocol. Section 4 presents the empirical results and ablation studies. Finally, Section 5 summarises the main findings, limitations, and future directions.

2 Problem Formulation and Related Work

2.1 Problem Definition

2.1.1 Game Description

Minesweeper is played on an $R \times C$ grid containing M hidden mines placed uniformly at random before the episode begins. At each turn the player selects an unrevealed cell: if it conceals a mine the game ends immediately in a loss; otherwise the cell is revealed, displaying the count of mines among its (up to eight) neighbours. Revealing a cell with count zero triggers a *cascade*: all connected zero-cells and their immediate numbered neighbours are opened automatically. The game is won when every non-mine cell has been revealed. Throughout this work we use the classic 9×9 board with $M = 10$ mines as the primary setting.

2.1.2 Why Minesweeper for Reinforcement Learning

Minesweeper offers four properties that make it a compelling RL benchmark.

Partial Observability. Mine locations are never directly observable; the agent infers danger solely from the adjacency numbers of revealed cells. The environment is therefore a Partially Observable Markov Decision Process (POMDP): the true state (full mine layout) is hidden, and the observation (the visible map) is an incomplete projection of it. This challenges value-based methods that assume full state access.

Sequential Decisions. Each click irrevocably changes the information state. Early moves that expose large safe regions provide far more information than late moves on a nearly complete board, so the quality of the *sequence* of decisions determines the outcome, not any single choice in isolation.

Structured Rewards. The reward signal is sparse by default—two terminal events (win/loss) separated by up to 71 uninstrumented steps on a 9×9 board. This creates a classic credit-assignment problem and motivates careful reward shaping, one of the central design questions we investigate in Section 3.1.2.

Deterministic Logic Within Stochastic Uncertainty. A crucial property distinguishing Minesweeper from purely stochastic games is that, given the visible map, some cells can be identified as *provably safe* or *provably dangerous* through constraint reasoning, even without knowing the full mine layout. An ideal agent should exploit this deterministic structure. We incorporate this insight directly into our feature representation (Section 3.2).

2.1.3 Formal RL Problem Statement

We model the task as an episodic POMDP $\langle \mathcal{S}, \mathcal{O}, \mathcal{A}, T, R, \gamma \rangle$:

- **Hidden state** $s \in \mathcal{S}$: the full board including mine locations, never directly observed.
- **Observation** $o \in \mathcal{O}$: the visible map $\mathbf{m} \in \{-1, 0, \ldots, 9\}^{R \times C}$, where -1 denotes a hidden cell, 0–8 a revealed adjacency count, and 9 an exploded mine.
- **Action space** $\mathcal{A}(o)$: the set of currently hidden cells, which shrinks as the board is revealed.
- **Transition** T: deterministic given the mine layout; stochastic from the agent's perspective because the layout is unknown.
- **Reward** R: the shaped reward function defined in Section 3.1.2.
- **Discount factor** $\gamma = 0.99$.

2.2 Related Work

Research on Minesweeper draws on both reinforcement learning and logical reasoning in puzzle games. On the RL side, Q-Learning and SARSA are foundational temporal-difference methods for learning action values from interaction [1,2], while Sutton and Barto's treatment of eligibility traces and linear function approximation motivates a compact, interpretable baseline when controlled ablation is more important than raw model capacity [3]. This is well suited to Minesweeper, whose sparse feedback and enormous effective state space make fully tabular methods impractical.

Minesweeper itself has long served as a benchmark for reasoning under uncertainty. Kaye showed that Minesweeper consistency is NP-complete [4], and classical solvers therefore rely on local logic, constraint satisfaction, and probabilistic guessing; Studholme's CSP formulation is a representative example [5]. Learning-based work ranges from relational approaches [6] to more recent CNN-based agents [7], suggesting that both symbolic structure and learned representations are useful in this domain.

Reward shaping and exploration are equally important in sparse-reward RL. Potential-based shaping can densify rewards without changing the optimal tabular policy [8], but in

practice shaping, exploration decay, and representation still interact strongly in finite data. Prior Minesweeper studies tend to emphasise either exact logic or higher-capacity models; by contrast, we isolate reward shaping, exploration schedules, and inference-oriented features within a shared linear-approximation pipeline.

3 Methodology

This section describes the three components of our system: the game environment (Section 3.1), the RL algorithms and feature representation (Section 3.2), and the training and evaluation framework (Section 3.3).

3.1 Environment Implementation

We implemented the Minesweeper environment from scratch as a self-contained module. The game rules and POMDP formulation are given in Section 2.1; this section focuses on the two key implementation decisions: the agent-facing API and the reward design.

3.1.1 Interface Design

The observation returned at each step is the visible map $\mathbf{m} \in \{-1, 0, \ldots, 9\}^{R \times C}$ (Section 2.1), together with auxiliary scalars (`done`, `win`, `trial_count`), packaged into a state dictionary. The environment exposes three methods following the standard Gym-style contract (Listing 1, Appendix 5).

A key design choice is **action masking**: `get_valid_actions()` returns only the currently hidden cells, so the agent's action space contracts as the board is revealed. This eliminates the possibility of selecting an already-open cell and keeps the learning signal uncontaminated by trivially invalid moves. When a safe cell with zero adjacent mines is clicked, a BFS cascade automatically opens all connected zero-cells and their numbered neighbours, exactly replicating standard Minesweeper rules. The number of newly revealed cells is returned so the reward function can scale accordingly.

In addition to the RL interface, the environment supports an interactive CLI mode and a Tkinter-based GUI for both human play and step-by-step agent replay, the latter being used extensively during debugging and qualitative analysis. Figure 4 in Appendix 5 shows screenshots of both interfaces.

3.1.2 Reward Design

The standard formulation of Minesweeper provides only two natural reward signals: $+1$ for winning and -1 for hitting a mine. On a 9×9 board with 10 mines, this leaves up to 71 consecutive steps with no feedback, creating a severe credit-assignment problem. Beyond the terminal rewards, we introduce two additional components to address this:

$$r_t = \begin{cases} +10.0 & \text{win (all safe cells revealed)} \\ -10.0 & \text{mine hit} \\ +0.05 \times k & \text{safe reveal: } k \text{ cells opened this step} \\ -0.5 & \text{repeat click on an already-revealed cell} \end{cases} \tag{1}$$

Dense progress signal ($+0.05 \times k$). Rather than waiting until game termination, the agent receives a small reward for each cell newly revealed in a step. Crucially, the reward scales with k—the number of cells opened by the BFS cascade—rather than being a flat per-action bonus. This alignment between reward and information gain has two effects: it provides a learning signal at every step instead of only at episode end, and it naturally incentivises moves that trigger large cascades early in the game, which are genuinely high-value actions. The magnitude 0.05 is chosen so that even a complete game of 71 safe reveals accumulates at most $71 \times 0.05 = 3.55$, keeping the terminal outcomes (± 10) strictly dominant.

Repeat-click penalty (-0.5). Action masking (Section 3.1.1) prevents the agent from selecting already-revealed cells during normal training, so this penalty is rarely triggered. Its purpose is to guard against degenerate behaviour if the masking is bypassed—for example, during manual testing or future modifications that relax the action filter. The value -0.5 is large enough to be discouraging but well below the mine penalty, preserving the ordering $r_{\text{lose}} \ll r_{\text{repeat}} < 0 < r_{\text{safe}} \ll r_{\text{win}}$.

3.2 Algorithm Implementation

This section describes the three algorithms evaluated in this work. SARSA(λ) serves as our primary method; Q-Learning and Monte Carlo are implemented as comparison baselines. All three share the same feature representation, exploration schedule, and action masking scheme described below.

3.2.1 Value Function Approximation

The Minesweeper state space is astronomically large. On a 9×9 board, each cell can take one of 11 values $\{-1, 0, 1, \ldots, 9\}$, yielding a theoretical state space of size $11^{81} \approx 10^{84}$. Storing a separate Q-value for every (state, action) pair—as classical tabular methods do—is therefore computationally infeasible. To address this, all three algorithms share a common **linear function approximation** scheme:

$$Q(s, a) \approx \mathbf{w}^\top \boldsymbol{\phi}(s, a), \tag{2}$$

where $\boldsymbol{\phi}(s, a) \in \mathbb{R}^{10}$ is a hand-crafted feature vector and $\mathbf{w} \in \mathbb{R}^{10}$ is a weight vector learned during training. A key limitation of linear approximation is that the model cannot discover combinatorial relationships among inputs on its own; any such interaction must be explicitly encoded in $\boldsymbol{\phi}$. This places the burden of representational expressiveness squarely on feature engineering, described next.

3.2.2 Feature Representation

Each feature vector $\boldsymbol{\phi}(s, a) \in \mathbb{R}^{10}$ is constructed for a candidate cell $a = (r, c)$ given the current observation s. Let $\mathcal{N}(r, c)$ denote the set of at most eight cells adjacent to (r, c), and let $H(r, c)$ and $\mathcal{D}(r, c)$ denote the subsets of hidden and revealed numbered cells in $\mathcal{N}(r, c)$, respectively. The ten features are summarised in Table 1.

Features f_1–f_4 capture local geometry around the candidate cell, ranging from raw neighbour counts to a normalised danger ratio; f_5–f_6 provide global board context; and f_7 is a fixed bias term that allows the linear model to learn a non-zero intercept.

Table 1: Feature vector $\boldsymbol{\phi}(s, a)$ for candidate cell $a = (r, c)$.

#	Name	Definition
f_1	`hidden_neighbor_ratio`	$\lvert H(r,c)\rvert \,/\, 8$
f_2	`neighbor_number_sum`	$\big(\sum_{n\in\mathcal{D}(r,c)} \mathrm{val}(n)\big) \,/\, 64$
f_3	`danger_ratio`	$\big(\sum_{n\in\mathcal{D}(r,c)} \mathrm{val}(n)\big) \,/\, \big(\max(\lvert H(r,c)\rvert, 1) \times 8\big)$
f_4	`is_isolated`	$\mathbf{1}$[no revealed cell exists in $\mathcal{N}(r,c)$]
f_5	`global_unrevealed_ratio`	(hidden cells on board) / $(R \times C)$
f_6	`global_mine_ratio`	(exploded mines visible on board) / $(R \times C)$
f_7	`bias`	Fixed constant 1.0
f_8	`inferred_safe`	$\mathbf{1}[(r,c) \in \mathcal{S}_{\mathrm{inf}}]$: provably safe by constraint inference
f_9	`inferred_mine`	$\mathbf{1}[(r,c) \in \mathcal{M}_{\mathrm{inf}}]$: provably a mine by constraint inference
f_{10}	`continuous_danger`	$\max_{n\in\mathcal{D}(r,c)} \frac{\mathrm{val}(n) - \lvert\mathcal{M}_{\mathrm{inf}}\cap\mathcal{N}(n)\rvert}{\lvert H(n)\rvert - \lvert\mathcal{M}_{\mathrm{inf}}\cap\mathcal{N}(n)\rvert}$, or 0.5 if $\mathcal{D}(r,c) = \emptyset$

Features f_8–f_{10}: constraint-based inference. These features encode the deterministic reasoning structure of Minesweeper: if a numbered cell's value equals its hidden-neighbour count, all those neighbours are mines; if its mine budget is fully accounted for, the remaining hidden neighbours are safe. f_8 and f_9 flag whether the candidate cell is provably safe or provably a mine under these rules. f_{10} extends this to a continuous danger estimate by taking the maximum remaining-mine fraction across all numbered neighbours.

3.2.3 Baseline Algorithms

To contextualise our approach, we implement three baselines spanning the main axes of temporal-difference learning: on- vs. off-policy updates, and bootstrapping vs. full returns.

SARSA(λ). SARSA(λ) is an on-policy TD algorithm that augments one-step SARSA with **eligibility traces**, distributing credit backward through the trajectory at every update. At each step t, the TD error is

$$\delta_t \;=\; \begin{cases} r_{t+1} - \mathbf{w}^\top \boldsymbol{\phi}(s_t, a_t) & \text{if } \textit{done}, \\ r_{t+1} + \gamma\, \mathbf{w}^\top \boldsymbol{\phi}(s_{t+1}, a_{t+1}) - \mathbf{w}^\top \boldsymbol{\phi}(s_t, a_t) & \text{otherwise}, \end{cases} \tag{3}$$

where a_{t+1} is the action *actually selected* under the current ε-greedy policy. The trace and weight updates are

$$\mathbf{e}_t = \gamma\lambda\, \mathbf{e}_{t-1} + \boldsymbol{\phi}(s_t, a_t), \qquad \mathbf{w} \leftarrow \mathbf{w} + \alpha\, \delta_t\, \mathbf{e}_t, \tag{4}$$

with $\mathbf{e}$ reset to $\mathbf{0}$ at episode start and $\mathbf{w}$ preserved across episodes. Default hyperparameters: $\alpha = 0.01$, $\gamma = 0.99$, $\lambda = 0.80$, $\varepsilon_0 = 0.30$, $\varepsilon_{\min} = 0.05$.

Two properties make SARSA(λ) well-suited to Minesweeper. On-policy updates ensure that Q-values reflect the true cost of ε-greedy exploration—including accidental mine clicks—yielding conservative estimates appropriate for this high-penalty environment. Eligibility traces additionally address the credit-assignment problem: a single cascade can determine the game's outcome, yet one-step TD propagates that signal back by only one step per episode; traces distribute it across the full preceding trajectory with exponential decay λ.

Q-Learning. Q-Learning is an *off-policy* TD algorithm that bootstraps from the *greedy* next action rather than the one actually taken:

$$\delta_t^Q = r_{t+1} + \gamma \max_{a' \in \mathcal{A}(s_{t+1})} \mathbf{w}^\top \boldsymbol{\phi}(s_{t+1}, a') - \mathbf{w}^\top \boldsymbol{\phi}(s_t, a_t), \qquad \mathbf{w} \leftarrow \mathbf{w} + \alpha\, \delta_t^Q\, \boldsymbol{\phi}(s_t, a_t). \tag{5}$$

This optimistic target converges to the optimal policy under perfect future behaviour, but tends to overestimate values in partially observable settings and is less stable during early high-exploration training. Q-Learning carries no eligibility trace.

Monte Carlo. Monte Carlo (MC) is an *on-policy*, *non-bootstrapping* method that defers all updates to episode termination, computing the full discounted return:

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k+1}, \qquad \mathbf{w} \leftarrow \mathbf{w} + \alpha\big(G_t - \mathbf{w}^\top \boldsymbol{\phi}(s_t, a_t)\big)\boldsymbol{\phi}(s_t, a_t). \tag{6}$$

Using an every-visit update, MC avoids bootstrapping bias entirely, but its episode-level updates introduce high variance and preclude any within-episode refinement—a practical disadvantage relative to the step-by-step TD methods above.

3.2.4 Exploration and Action Masking

Exploration. All three algorithms use ε-greedy action selection. At the end of each episode, ε is multiplied by a decay factor of 0.9995 and floored at $\varepsilon_{\min} = 0.05$. Random actions are drawn uniformly from $\mathcal{A}(s)$—the set of currently hidden cells—ensuring that exploratory moves never repeat an already-revealed cell.

Action masking. At every decision step, the agent selects only from $\mathcal{A}(s)$, the list of currently hidden cells returned by `env.get_valid_actions()`. This separation of concerns—the environment enforces legality, the algorithm enforces optimality within legal moves—ensures that Q-value estimates are never distorted by repeat-click penalties during normal training.

3.3 Training and Evaluation Framework

All experiments use the environment, reward function, and feature space described in Sections 3.1, 3.2, and 3.2.2. Unless otherwise noted, the primary benchmark is the standard 9×9 board with 10 mines. Each reported configuration is evaluated over three random seeds, and we report mean $\pm$ standard deviation when multiple runs are available.

We track four complementary metrics: overall win rate, first-click death rate (FCDR), conditional win rate, and average number of decision steps per episode. For difficulty-scaling experiments, we also report episode time.

The FCDR problem we mention above is because first action of every episode is taken on a completely hidden board, giving no basis for differentiation between cells. For the 9×9 grid with 10 mines, the theoretical first-click death probability is $10/81 \approx 12.3\%$—irreducible noise that no algorithm can eliminate. A first-click death (FCD) is recorded when an episode terminates in a loss after exactly one selected action. Because this event occurs before the agent has observed any informative clue, it reflects unavoidable board randomness rather than informed policy failure. To separate this source of noise from later decision quality, we define

$$\text{FCDR} = \frac{N_{\text{first-click deaths}}}{N_{\text{episodes}}}, \qquad \text{Conditional Win Rate} = \frac{N_{\text{wins}}}{N_{\text{episodes}} - N_{\text{first-click deaths}}}. \tag{7}$$

We report both the overall win rate and the conditional win rate (Section 3.3), the latter excluding first-click deaths to isolate policy quality from unavoidable opening randomness.

The empirical study is organised as four targeted ablations. First, we test a 2×2 exploration design that combines cross-episode ε decay with an in-episode adaptive schedule based on board progress. Second, we vary the eligibility-trace parameter λ for SARSA(λ) while keeping the exploration policy fixed. Third, we evaluate transfer across board sizes with similar mine densities. Finally, we remove selected dimensions of the feature vector to assess which components contribute meaningfully to performance.

4 Comparison and Analysis

4.1 Exploration Strategy Ablation

The epsilon ablation evaluates a 2×2 design: whether exploration decays across episodes, and whether it adapts within each episode based on the fraction of opened cells. The results are shown in Table 2.

Table 2: Epsilon-strategy ablation on the $9 \times 9/10$ board. Values are percentages.

Strategy	Decay	Adaptive	Win Rate	Conditional WR
Fixed	No	No	56.33 ± 6.96	64.22
Decay	Yes	No	66.47 ± 0.80	75.51
Adaptive	No	Yes	67.27 ± 0.17	76.50
Decay+Adaptive	Yes	Yes	$\mathbf{69.43} \pm 0.41$	**79.48**

The two components control exploration from different perspectives.

Decay policy comes from model perspective. We assume that as training proceeds, the model will learn a better policy. Thus, the agent gradually reduces the base exploration rate between episodes, and shifts from exploration to exploitation:

$$\varepsilon_{\text{base}} \leftarrow \max(\varepsilon_{\min},\ 0.9995\,\varepsilon_{\text{base}}).$$

Adaptive comes from game perspective. It adjusts exploration within a single game according to how much of the board has already been opened:

$$\varepsilon_{\text{local}} = \varepsilon_{\text{base}}(1 - \text{opened\_ratio})^2.$$

This makes early-game exploration possible while suppressing risky random moves in the late game. We therefore choose Decay+Adaptive as the final exploration strategy because it combines long-term training decay with board-state-dependent caution. Furthermore, it has the best performance.

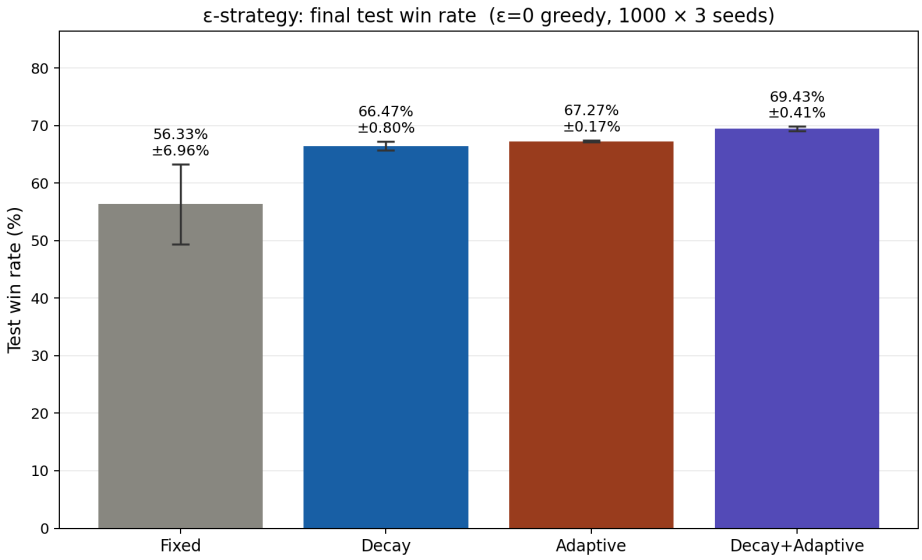


Figure 1: Epsilon-strategy ablation on the $9 \times 9/10$ board. Win rate is reported as mean $\pm$ standard deviation across three seeds.

4.2 Algorithm Comparison

We also compare the three implemented learning algorithms under the same feature representation and Decay+Adaptive exploration strategy. Table 3 shows that SARSA(λ), Q-Learning, and Monte Carlo achieve similar final performance on the $9 \times 9/10$ board.

Table 3: Algorithm comparison under Decay+Adaptive exploration. Values are percentages except for steps.

Algorithm	Win Rate	Conditional WR	Avg. Steps
SARSA(λ)	69.43 ± 0.41	79.48 ± 0.32	14.0 ± 0.3
Q-Learning	69.93 ± 0.12	79.26 ± 0.61	14.1 ± 0.2
Monte Carlo	68.83 ± 1.44	78.16 ± 1.72	14.1 ± 0.2

The three algorithms differ in their update targets, but their final performance is close once the feature representation and exploration policy are fixed. This suggests that, in our implementation, exploration strategy and feature design have a larger practical effect than the choice among these three classical learning rules.

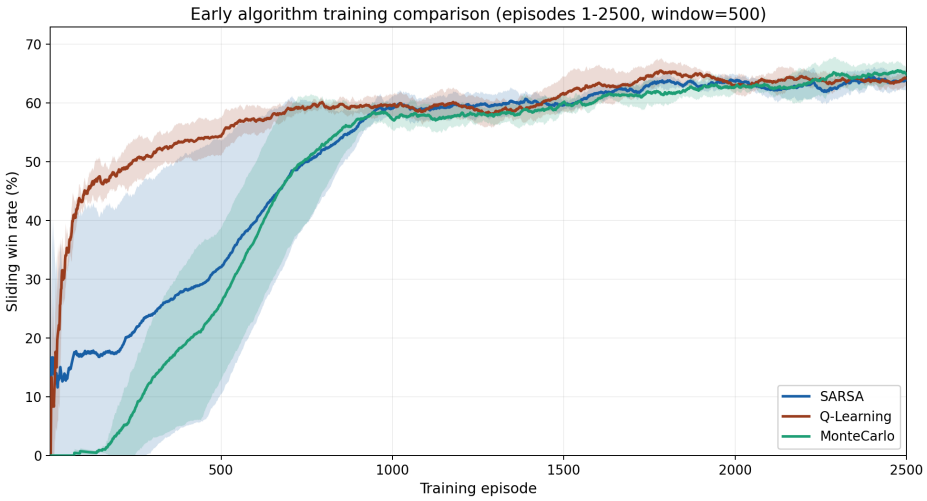


Figure 2: Training curves for SARSA(λ), Q-Learning, and Monte Carlo under the same Decay+Adaptive exploration strategy.

Although the final win rates are similar across the three algorithms, their early-stage learning curves show **different convergence speeds**. This suggests that we need to evaluate from diverse dimensions.

4.3 Lambda Ablation

Table 4 studies the effect of the eligibility-trace parameter in SARSA(λ). The values $\lambda = 0.0$, 0.4, and 0.8 perform **similarly**, while $\lambda = 1.0$ is less stable and gives a lower conditional win rate.

Table 4: SARSA(λ) ablation under Decay+Adaptive exploration.

λ	Win Rate	Conditional WR	Avg. Steps
0.0	69.50 ± 0.29	79.37 ± 0.57	14.0 ± 0.2
0.4	68.93 ± 0.66	79.08 ± 0.08	13.9 ± 0.2
0.8	69.43 ± 0.41	79.48 ± 0.32	14.0 ± 0.3
1.0	67.83 ± 1.45	77.39 ± 2.09	14.1 ± 0.1

4.4 Difficulty Scaling

To test whether the best configuration transfers across board sizes, we evaluated SARSA($\lambda = 0.8$) with Decay+Adaptive exploration on three boards with mine densities close to 12%. The results are shown in Table 5.

Table 5: Difficulty comparison across board sizes.

Board	Win Rate	Cond. WR	FCDR	Avg. Steps	Time (ms)
4×4, 2 mines	76.00 ± 2.89	86.97 ± 2.46	12.63 ± 1.31	3.7 ± 0.1	0.31 ± 0.01
6×6, 5 mines	62.47 ± 0.93	72.45 ± 1.15	13.77 ± 1.04	7.2 ± 0.0	1.52 ± 0.03
9×9, 10 mines	69.43 ± 0.41	79.48 ± 0.32	12.63 ± 0.85	14.0 ± 0.3	8.89 ± 0.18

The 4×4 board is the easiest, as expected. The lower performance on the 6×6 board is plausible because its mine density ($5/36 \approx 13.9\%$) is slightly higher than that of the 9×9 setting ($10/81 \approx 12.3\%$), and the smaller board provides fewer revealed-number patterns for inference.

4.5 Feature Ablation

The feature ablation removes selected low-impact dimensions from the feature vector and retrains the agent. Table 6 shows that dropping f_3 and f_6 does not meaningfully change performance; the small increase in conditional win rate is within the run-to-run variation across seeds.

Table 6: Feature ablation results for SARSA($\lambda = 0.8$) with Decay+Adaptive exploration.

Variant	Win Rate	First-Click Death	Conditional WR
All features	68.63 ± 2.31	12.03 ± 0.74	78.02 ± 2.42
Drop f_3	68.20 ± 0.96	12.87 ± 0.34	78.28 ± 1.37
Drop $f_3 + f_6$	68.20 ± 0.96	12.87 ± 0.34	78.28 ± 1.37

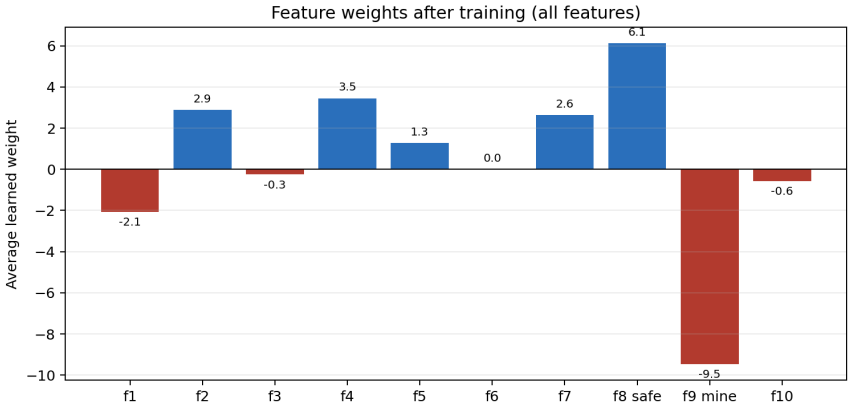


Figure 3: Average learned feature weights for the full-feature SARSA(λ) agent.

The learned weights provide a useful sanity check: inferred-safe cells receive a strong positive weight, while inferred-mine cells receive a strong negative weight. This also explains the ablation result: the new inference-based features likely **encode much of the risk information** that f_3 and f_6 were meant to provide.

4.6 Discussion

Three findings stand out. First, **exploration strategy** has the largest practical effect in our experiments: Decay+Adaptive exploration improves performance by roughly 13 percentage points over fixed exploration on the primary benchmark. Once the exploration policy is fixed, the choice among SARSA(λ), Q-Learning, and Monte Carlo has a smaller effect on final win rate.

Second, **feature engineering** is crucial. The feature ablation shows that dropping f_3 and f_6 does not meaningfully reduce performance, likely because the new inference-based features already encode much of the same risk information in a more direct form. The learned weights further support this interpretation: inferred-safe cells receive a strong positive weight, while inferred-mine cells receive a strong negative weight.

Third, **conditional win rate** is necessary for fair evaluation. Around 12–14% of games are lost on the first click before any useful board information is available, so reporting conditional win rate helps separate unavoidable opening randomness from policy quality.

5 Conclusion

This project studied Minesweeper as a partially observable reinforcement learning problem with both stochastic uncertainty and deterministic logical structure. On the classic 9×9 board with 10 mines, the best configuration achieved a 69.43% overall win rate and a 79.48% conditional win rate, showing that a carefully designed linear agent can solve a large fraction of standard boards.

Overall, the experiments show that careful evaluation design is as important as the learning rule itself. Adaptive exploration, conditional win-rate reporting, and inference-based features together made the final policy more stable and more interpretable.

There are still several limitations. The current model uses linear function approximation covered in our lectures, so it cannot automatically discover complex spatial patterns beyond the hand-crafted features. Future work could use deeper function approximators such as DQN or convolutional networks, and test transfer to larger board sizes.

References

[1] G. A. Rummery and M. Niranjan, "On-line Q-learning using connectionist systems," University of Cambridge, Department of Engineering, Tech. Rep. CUED/F-INFENG/TR 166, 1994.

[2] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, no. 3–4, pp. 279–292, 1992.

[3] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

[4] R. Kaye, "Minesweeper is NP-complete," *The Mathematical Intelligencer*, vol. 22, no. 2, pp. 9–15, 2000.

[5] C. Studholme, "Minesweeper as a constraint satisfaction problem," University of Toronto, Tech. Rep., 2001.

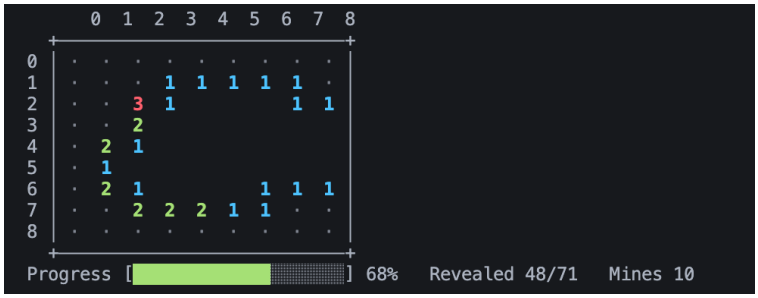
[6] L. Peña Castillo and S. Wrobel, "Learning minesweeper with multirelational learning," in *Proceedings of the 18th International Joint Conference on Artificial Intelligence (IJCAI)*, 2003, pp. 533–540.

[7] W. Wang and C. Lei, "Training a minesweeper agent using a convolutional neural network," *Applied Sciences*, vol. 15, no. 5, p. 2490, 2025.

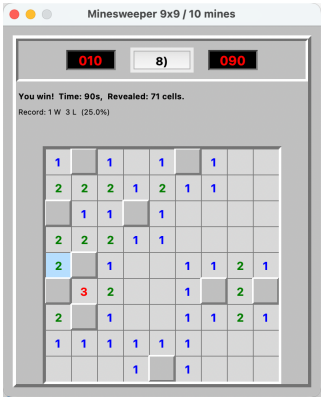
[8] A. Y. Ng, D. Harada, and S. J. Russell, "Policy invariance under reward transformations: Theory and application to reward shaping," in *Proceedings of the 16th International Conference on Machine Learning (ICML)*, 1999, pp. 278–287.

Appendix A: Environment Details

This appendix provides supplementary details on the Minesweeper environment, including screenshots of both interactive interfaces and the agent-facing API.



(a) CLI mode: ASCII board with coordinate input.



(b) GUI mode: Tkinter window with mine counter and timer.

Figure 4: The two interactive interfaces provided by the environment. Left: CLI mode for debugging and headless use. Right: Tkinter GUI supporting human play and autonomous agent replay with per-step action highlighting.

The environment exposes three methods following the standard Gym-style contract:

```
state  = env.reset()                              # begin new episode
valid  = env.get_valid_actions()                  # list of hidden cells (r, c)
state, reward, done, info = env.step((r, c)) # reveal cell (r, c)
```

Listing 1: Agent-facing API of `MinesweeperEnv`.